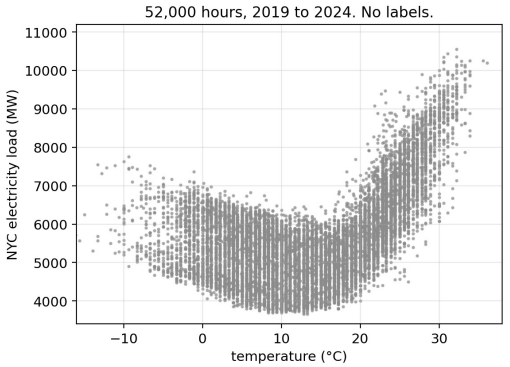
Learning without labels

- **Supervised learning needs a target.** Every sample comes with the answer, and the model learns to reproduce it
- **Unsupervised learning has no target.** The method only sees the inputs. It has to find the structure on its own
- **Week 7 reduced the number of features.** This week groups the samples
- **Three jobs today.** Put similar samples in groups, describe where the data are dense, and flag the samples that sit where nothing else does
- **No answer key.** A clustering is a description of the data, not a discovery. You judge it with things the algorithm never saw

Group
Climate zones from 24 numbers per grid cell, with no map of zones to copy. That is the assignment.

Describe
Where are the data dense? A few bell curves can describe a cloud of points and say how typical each point is.

Flag
Hours when the New York power grid did something it almost never does. Found without telling the model what unusual means.



This is all a clustering method sees. Hourly electricity demand in New York City against temperature, six years. No labels, no seasons, no weekday flag. Everything that follows starts from a picture like this one.

Where clustering shows up in climate science

Climate zones
Köppen drew his zones by hand from temperature and rain thresholds in 1884. A clustering draws them from the data. The assignment.

Weather regimes
Winter days over the North Atlantic fall into a few recurring pressure patterns. Forecasters use them for the two to four week range.

Storm genesis regions
Where do tropical cyclones form? The dense patches of starting points are the basins.

Air mass and source types
Aerosol or back-trajectory samples cluster into source regions: dust, smoke, sea salt, traffic.

Sensor behavior
A cheap sensor behaves differently in different weather. A mixture model finds the regimes and calibrates each one. Slide 13.

Anomalies
Faulty instruments, extreme days, odd samples. The points that belong to no group.

- **The common thread:** the groups come from the data, not from a person drawing lines on a map
- **The common trap:** the groups also depend on choices the person made first. Which features, how they are scaled, how many groups
- **So the rule for this week:** say what you chose and why, then check the result against something the algorithm never saw

Similar means close. You decide what close means

- **Every method today works on distances between samples.** A sample is a row, a feature is a column, and distance is measured across the columns
- **Raw units decide the distance.** New York demand is in thousands of megawatts, temperature in tens of degrees. Unscaled, temperature barely counts
- **Standardize, or scale to a fixed range.** StandardScaler gives each column mean 0 and spread 1. MinMaxScaler squeezes each column into a range like −1 to 1
- **Take logs of things that span orders of magnitude.** Income, emissions, rainfall. A doubling should count the same at the bottom and the top
- **For a map, each grid cell is a sample.** Its features are its values through the year. Scale all months together, so January and July stay comparable. Slide 7
- **The features you leave out cannot form a cluster.** No fuel mix column, no coal cluster. The demo shows this

```
from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(feat)

from sklearn.preprocessing import MinMaxScaler
X = MinMaxScaler((-1, 1)).fit_transform(feat)
```

Why this matters more than in Week 5

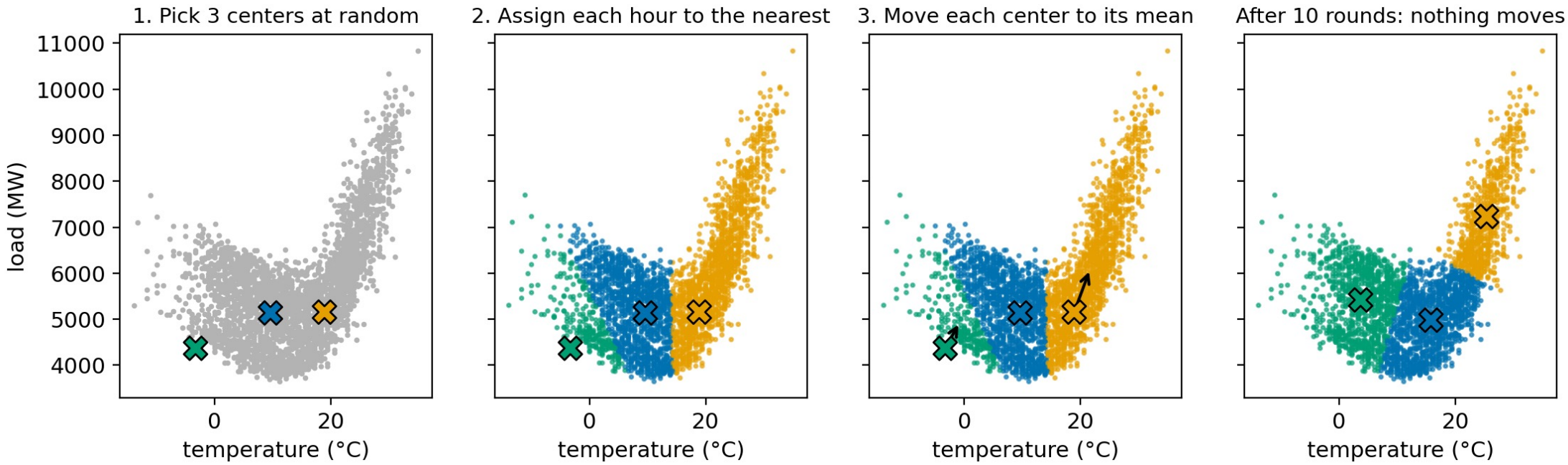
A random forest splits one feature at a time, so units never mattered. Distance mixes all the features into one number, so units are the whole game.

Scaling is a modeling choice

Standardizing says every feature deserves equal weight. That is a claim about the problem. Sometimes it is wrong, and you weight a feature on purpose.

Tutorial, section 1, does the log and standardize steps on the country data. The assignment, Part 2, uses MinMaxScaler on the climate normals

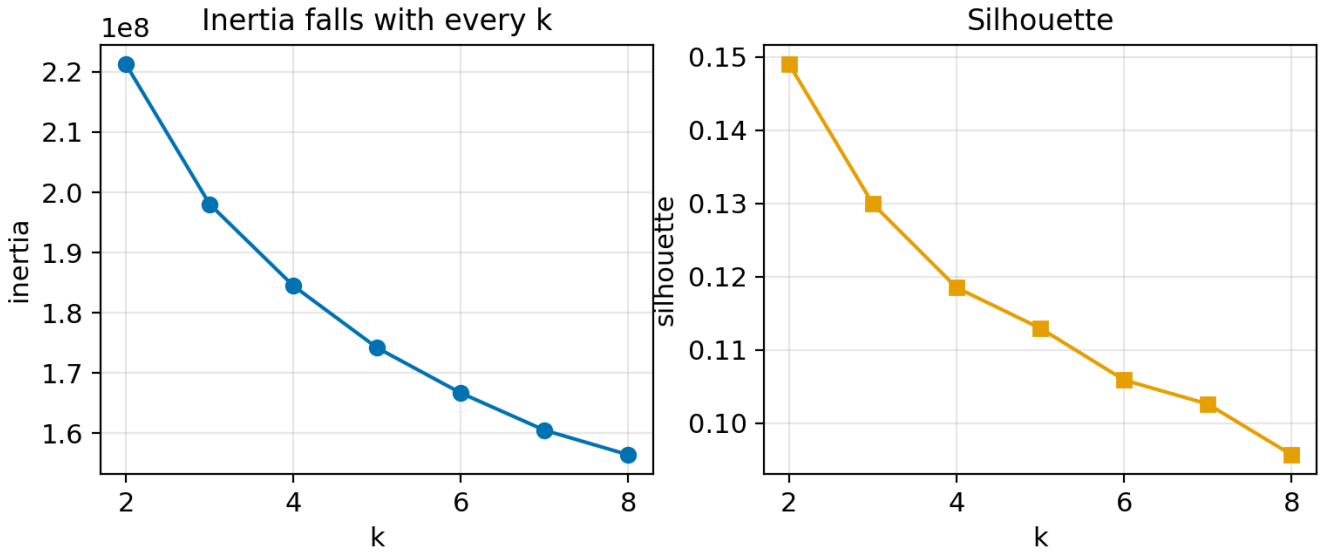
k-means in four steps



- **Pick k centers at random. Assign each sample to the nearest center. Move each center to the mean of its samples. Repeat until nothing moves.**
- **It always finishes, but not always in the same place.** Different starts give different answers. scikit-learn runs it 10 times from different starts and keeps the best, the n_init argument
- **What it assumes:** round clusters of similar size, and that every sample belongs somewhere. It will split a smooth curve into chunks, as it does here
- **What it gives you:** a label for every sample, and k centers you can read in the original units. Always translate the centers back

3,000 hours of New York demand against temperature, standardized before clustering, plotted in original units. k = 3. Centers are the crosses

How many clusters? Two diagnostics, one decision



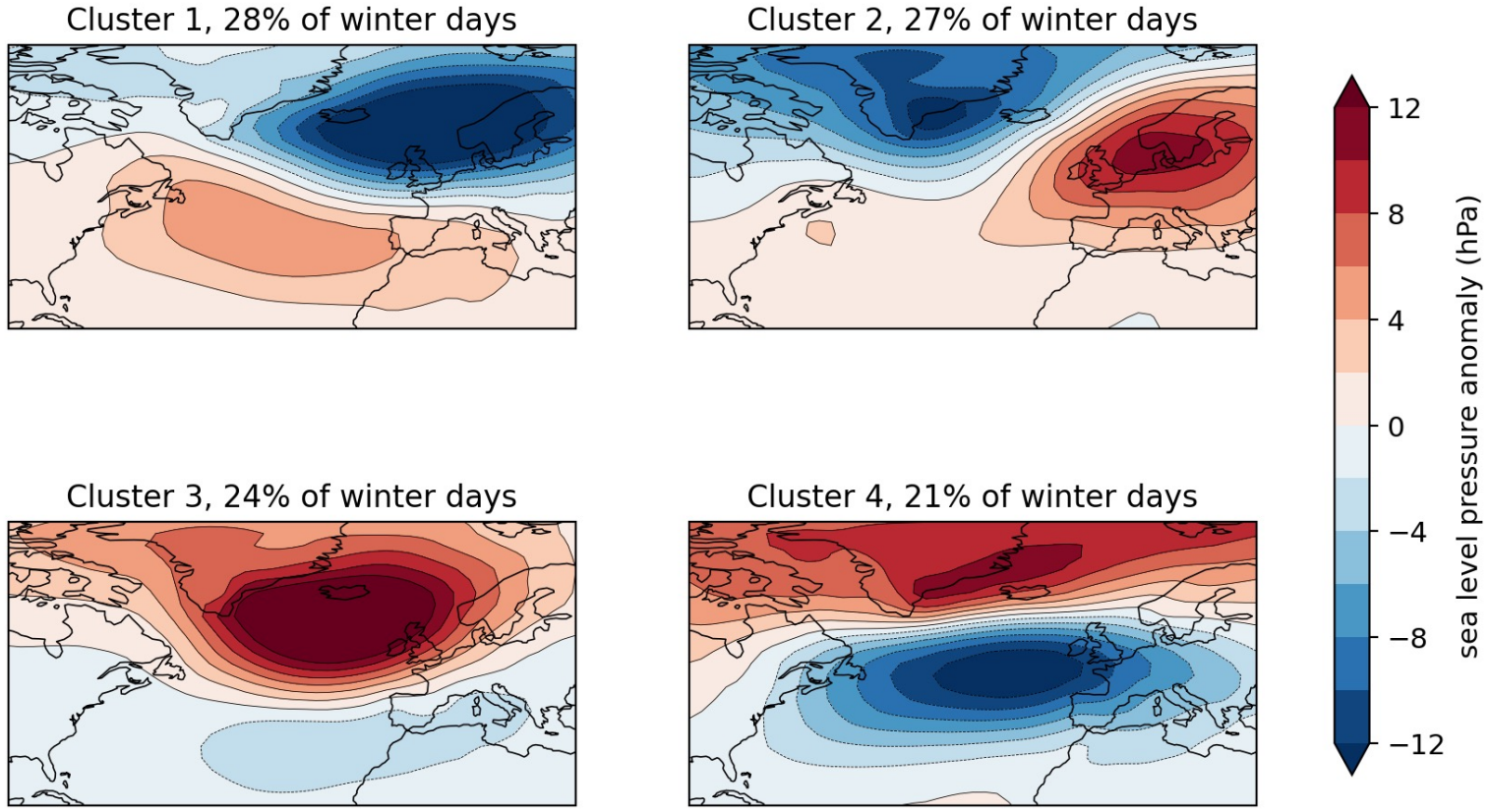
What to do. Use the curves to rule out silly values. Then choose k for a reason you can state: the number of regimes in the literature, the number of zones a map can show, what the next step needs. Report the choice and the reason. Try k plus or minus one and say whether the picture changes.

What not to do. Scan k from 2 to 20, pick the highest silhouette, and call it the number of regimes in the atmosphere.

- **k is an input.** The algorithm never tells you the right one
- **Inertia** is the total squared distance from samples to their centers. It always falls as k grows, so look for the bend where it stops falling fast
- **Silhouette** compares each sample's distance to its own cluster with its distance to the nearest other cluster. Near 1 is tight and separate. Near 0 is overlap
- **Here, on winter pressure maps:** no sharp bend, and silhouettes near 0.15 at best. That is normal for real data. The days lie on a continuum
- **Stability is the third check.** Four clusters from eight single random starts agree with each other at Rand index 0.30 to 0.97. If the answer depends on the seed, it is not a finding

Winter daily sea level pressure anomalies over the North Atlantic, 1979-2024, 4,152 days, 1225 grid cells reduced to 14 PCs before k-means. Silhouette on a 3,000-day sample

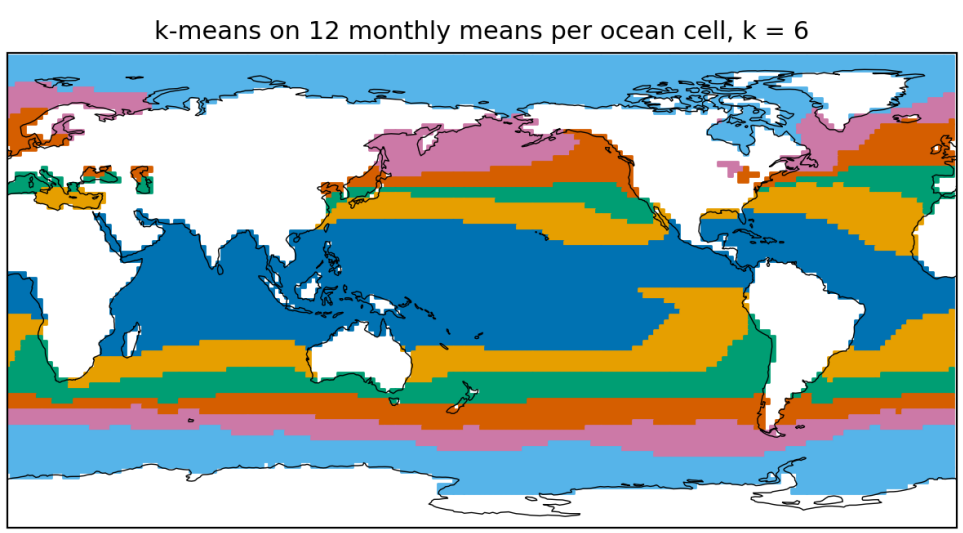
Application: winter weather regimes

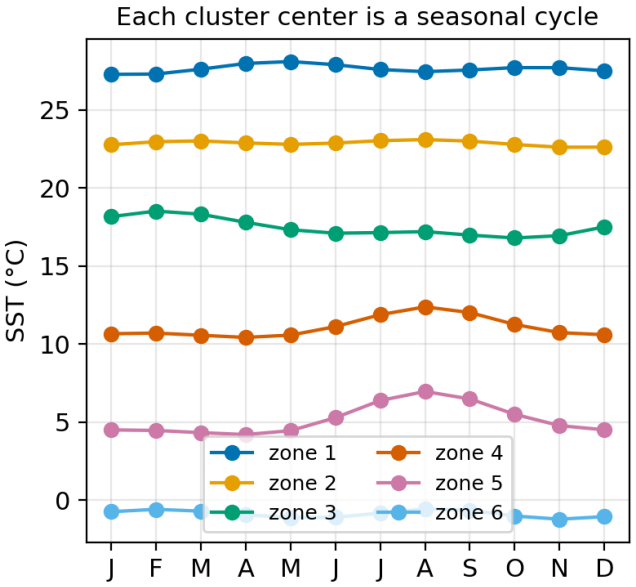


- **The data:** daily sea level pressure, December to February, 1979-2024, from the NCEP reanalysis. One sample is one day's map over the North Atlantic and Europe
- **The steps:** remove the seasonal cycle, weight by area, PCA down to 14 components, k-means with k = 4 and 100 starts
- **Three of the four are textbook regimes:** NAO positive (28% of days), Scandinavian blocking (27%), NAO negative (24%). The fourth, a Greenland high (21%), is a cousin of NAO negative. The textbook Atlantic ridge did not come out on its own here
- **Why anyone cares:** a day's regime lasts about 4 days on average and sets the weather downstream. Cold spells in Europe, wind power output, where storms track. Forecast centers issue regime probabilities for weeks two to four
- **The caution from the last slide applies.** Four is a choice from the literature, and the silhouette does not single it out. Single random starts reach inertias within 0.3% of each other and still disagree on a third of the days

Composite anomaly of all days in each cluster, hPa. Method follows Michelangeli, Vautard and Legras 1995. Regime names after Cassou 2008

The assignment skill: a map is a table of samples





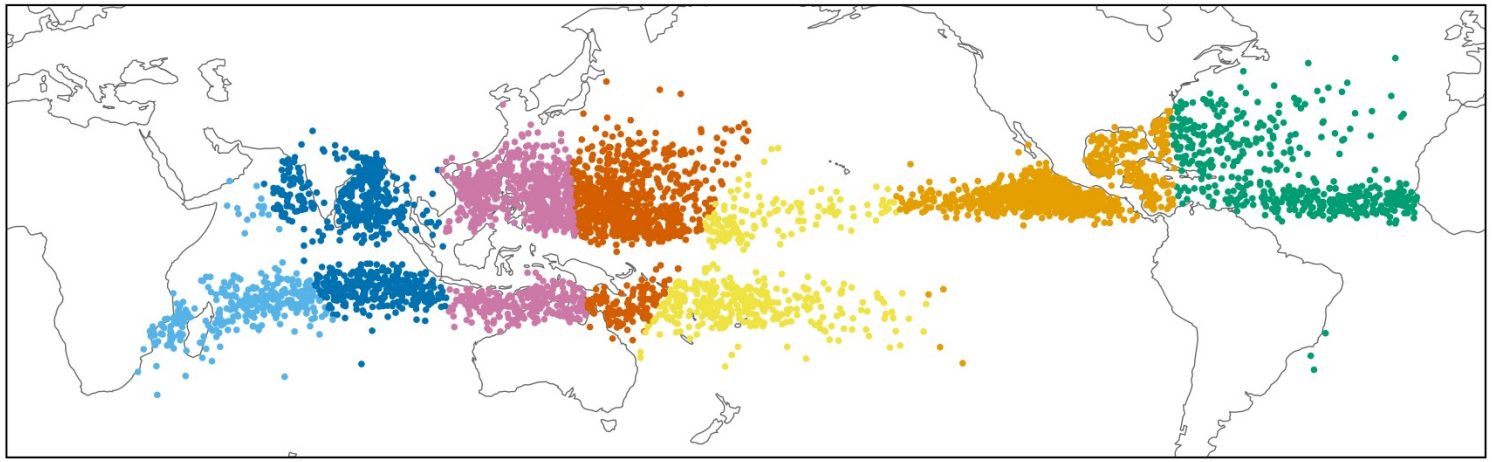

```
vals = clim.values          # 12 x lat x lon
ocean = ~np.isnan(vals[0])  # True where data
feat = vals[:, ocean].T     # samples x 12
lat_m = lat_grid[ocean]
lon_m = lon_grid[ocean]
sc = MinMaxScaler((-1, 1))
X = sc.fit_transform(
    feat.reshape(-1, 1)).reshape(feat.shape)
lab = KMeans(6, n_init=20).fit_predict(X)
plt.scatter(lon_m, lat_m, c=lab, s=4,
            cmap=ListedColormap(colors))
```

- **Each ocean cell is a sample with 12 features, its monthly mean temperatures.** Land cells are dropped with a mask, not filled
- **One MinMaxScaler for all 12 months.** Reshape to one column, scale, reshape back, so the seasons survive
- **The map is a scatter plot of the kept cells, colored by label.** A discrete colormap, one color per cluster
- **Read the centers.** Each one is a seasonal cycle. Zones follow latitude, except where currents bend them

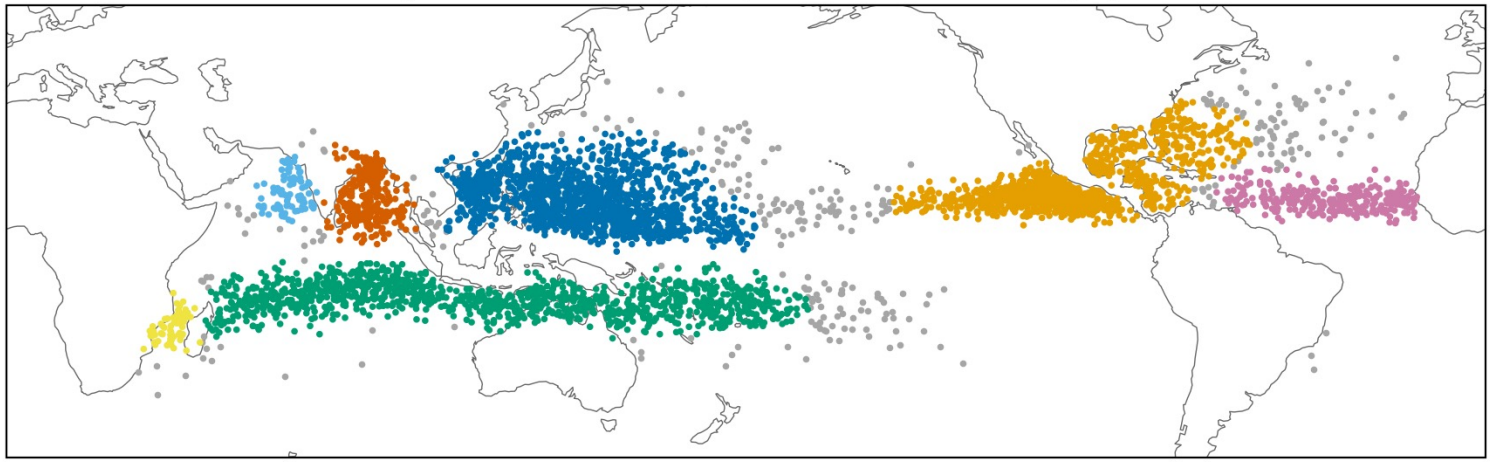
NOAA ERSST v5 monthly climatology 1991 to 2020, 2 degree grid, 10,988 ocean cells, k = 6. The assignment does the same with 12 months of temperature and 12 of precipitation over the United States

DBSCAN: density instead of centers

k-means, k = 7: every storm gets a cluster



DBSCAN, eps = 500 km, min_samples = 25: 7 clusters, gray = noise



- **Two knobs and no k.** A radius, eps, and a count, min_samples
- **A core point has at least min_samples neighbors within eps.** Cores that can reach each other through other cores form one cluster. Points near a core but not cores themselves ride along
- **Everything else is noise, label −1.** Not a cluster, not a mistake. A point that belongs to no dense region
- **Clusters can be any shape.** A long curved band is one cluster if it is dense all the way along
- **Storm genesis points, 4,799 storms since 1980.** k-means with 7 clusters chops the oceans into pieces. DBSCAN with a 500 km radius finds 7 dense regions, the basins, and calls 310 storms noise

IBTrACS v4, first recorded position of each storm, 1980 to 2024. DBSCAN uses great-circle distance (metric="haversine"). Gray points in the lower panel are noise

DBSCAN: the radius does all the work

eps (km)	clusters	noise points
200	9	3,667
300	18	1,681
400	11	658
500	7	310
700	2	127

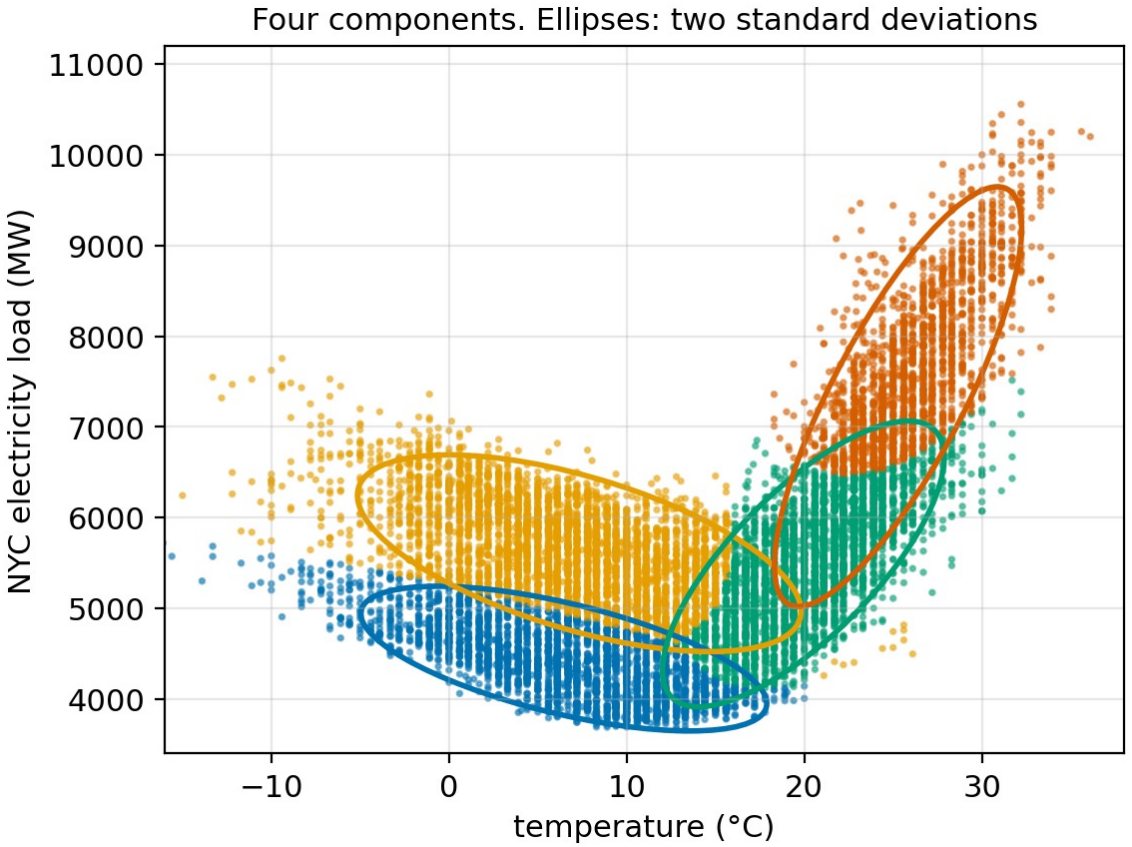
What the noise list held. 310 storms. Subtropical Atlantic storms that formed near 40°N, the sparse central Pacific, three South Atlantic storms including Catarina in 2004.

What it got wrong. It joined the eastern Pacific to the Gulf of Mexico across Central America, because the two are within 500 km and distance ignores land.

- **Too small and almost everything is noise.** Too large and everything is one cluster. In between, the number of clusters rises and falls
- **There is no inertia curve to look at.** A usual habit: set min_samples to about twice the number of features, then sort each point's distance to its min_samples-th neighbor and look for the knee
- **Varying density is the weak spot.** One eps cannot fit a dense basin and a sparse one at the same time. The South Pacific storms end up as noise because the region is sparse, not because the storms are odd
- **No predict method.** DBSCAN labels the data it was fit on and that is all. To label a new point, use a nearest neighbor classifier on the core points, or ask whether it falls within eps of any core
- **Cost.** Fast with a tree index when eps is small. Slow when eps is large or the data are high-dimensional, because then everything is near everything

min_samples = 25 throughout. The haversine metric needs latitude and longitude in radians and eps in radians, so 500 km is 500 / 6371

Gaussian mixture models: soft clusters

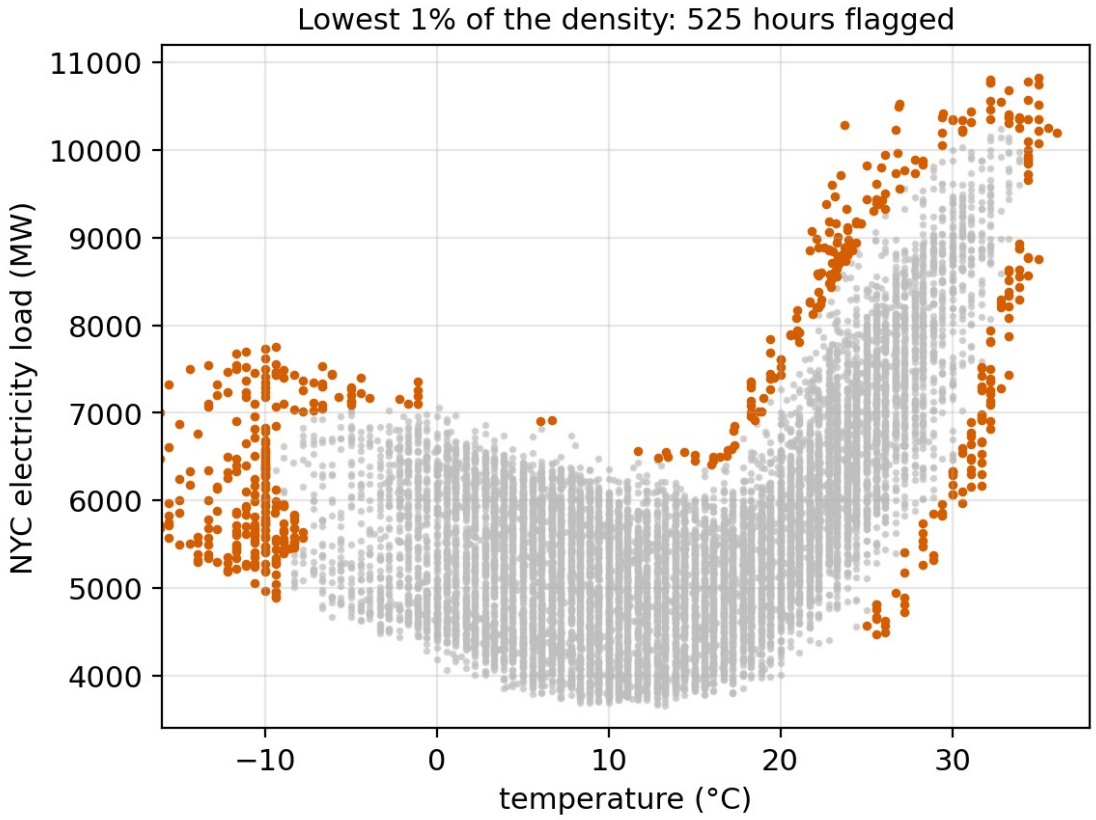



```
gm = GaussianMixture(n_components=4, n_init=5).fit(X)
labels = gm.predict(X);  probs = gm.predict_proba(X)
```

- **The idea:** the data were drawn from k bell curves, each with its own center, shape, and share. Fit the curves
- **Each ellipse is a component.** The center is a mean, the shape is a covariance, so a component can be long, thin, and tilted. k-means only has round clusters of one size
- **Membership is a probability.** Every hour gets a probability for each component. 11% of the hours here have no component above 0.6. They sit between two ellipses, and the model says so
- **The fit is by expectation maximization.** Guess the components, compute each point's responsibilities, refit the components from the responsibilities, repeat. k-means with soft assignments
- **What you get that k-means cannot give:** a density. For any point, the model says how likely a point like this is. The next slide uses that

Hourly New York City demand and temperature, 2019 to 2024. Four components, full covariance. Ellipses are two standard deviations. Colors are the most probable component

Density as an anomaly detector

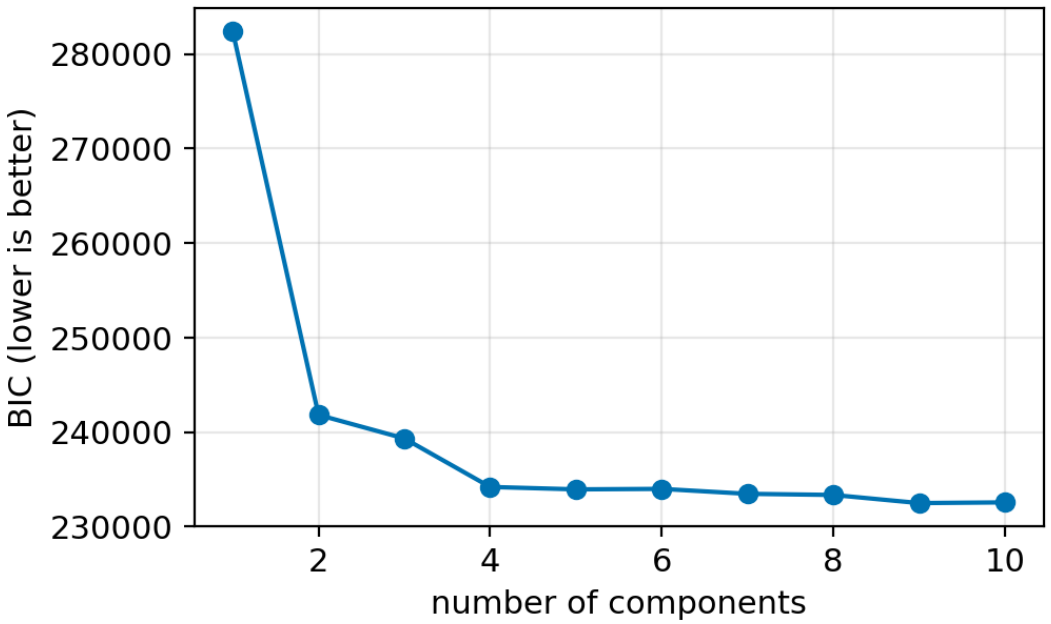


- **score_samples gives the log density at every point.** Rank the points. The lowest ones are where the model has seen almost nothing
- **Flag the lowest 1%, 525 hours.** The threshold is a choice, like k. Choose it from how many you can afford to inspect
- **What it found:** the polar vortex of late January 2019, the Christmas Eve 2022 cold snap, the July 2019 heat wave. The hours at the extreme tips of the cloud
- **What it did not find:** the spring 2020 lockdown, when demand fell well below normal for the temperature. Only 4 of those 1,340 hours were flagged. They sit inside the cloud, just low in it
- **The lesson:** an anomaly in one feature set is normal in another. The model flags rare combinations of the features you gave it, nothing more

```
dens = gm.score_samples(X)
flag = dens < np.percentile(dens, 1)
```

Flagged hours by day: 2019-01-21 (24), 2019-01-31 (24), 2022-12-24 (23), 2019-02-01 (21), 2019-01-22 (19), 2023-02-04 (12). Dates in local time

How many components? BIC, and a judgment call

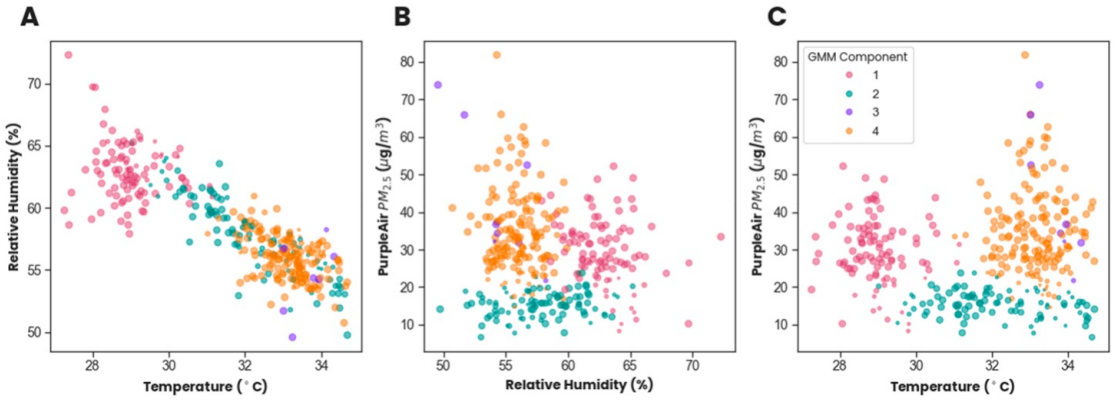


Bayesian mixtures. BayesianGaussianMixture takes an upper bound on the number of components and switches off the ones it does not need. Useful when you have no idea. It is not magic either: it has a prior that plays the role of the charge.

- **A mixture model is a probability model, so it has a likelihood.** More components always raise the likelihood. A criterion charges for the extra parameters
- **BIC = parameters × log(n) − 2 × log likelihood.** Lower is better. AIC is the same idea with a smaller charge, so it tends to pick more components
- **Here BIC keeps falling to 9 components.** The big drop ends at 4. After that each component buys a little, because the cloud is a curve and bell curves tile a curve better the more there are
- **So the honest answer is 4 with a reason,** not 9 because a number said so. Where the curve flattens is where you stop, unless the next step needs more
- **On the demo data, BIC picked 2.** Two components is the simplest story the countries support. The tutorial's four is a choice for interpretability

BIC for 1 to 10 components on the standardized New York data, 52,439 hours, n_init = 3

Application: calibrating a cheap PM2.5 sensor



Reading the figure. Each daily average is colored by its most probable component. Panel A, humidity against temperature: the components line up along the season. Panels B and C: component 2 (teal) is the low-pollution background, component 4 (orange) is the dry season with high readings. Point size is the membership probability. Small points are the days the model was unsure about.

What to take from it. The same four-component fit does clustering, regression, and uncertainty at once, because a mixture is a full probability model of the data. A random forest can only predict.

- **The problem:** a 250 dollar PurpleAir sensor next to a reference monitor at the US embassy in Accra, Ghana, March 2020 to March 2021. Raw agreement $R^2 = 0.53$. The sensor reads high in the humid season and low in the dry season
- **The method:** fit a Gaussian mixture to the joint data: sensor PM2.5, temperature, humidity, and reference PM2.5 together. BIC picked 4 components. Then predict the reference value as the mixture's conditional mean given the three inputs. That is Gaussian mixture regression
- **The result:** $R^2 = 0.88$ and mean error 2.2 µg/m³, against 0.81 for linear regression and 0.81 for a random forest on the same inputs
- **Why it worked:** the components are regimes of sensor behavior. Component 1 is the rainy season, high humidity and an overreading sensor. Component 4 is the Harmattan, hot, dusty, and underreading. One line cannot fit both. Four bell curves can
- **Two bonuses:** the model still works when an input is missing, and it gives an uncertainty for every corrected value. The 95% intervals held the reference value 96% of the time

McFarlane, Raheja, Malings, Appoh, Hughes and Westervelt 2021, ACS Earth and Space Chemistry 5, 2268. Figure 5, open access

Which method, when

	k-means	DBSCAN	Gaussian mixture
What you choose	k, the number of clusters	eps and min_samples, a density	k, the number of components, and the covariance shape
Cluster shape	Round, similar size	Any shape, if dense throughout	Ellipses of any tilt and size
Outliers	Forced into a cluster	Labeled −1, noise	Low density, you set the threshold
Membership	Hard, one label	Hard, one label or noise	Soft, a probability per component
New samples	predict	None. Use nearest neighbors on the cores	predict and predict_proba
Choosing the size	Elbow, silhouette, stability	Knee in the k-distance plot	BIC or AIC
Scales to	Millions of samples, MiniBatchKMeans	Large n if eps is small	Thousands to a few million, depends on covariance type
Use it for	A quick partition, compressing a field, a first look	Spatial data, odd shapes, finding noise	Overlapping groups, densities, anomalies, uncertainty

- **All three need scaled features and all three depend on their knobs.** None of them tells you whether the groups are real
- **The tutorial runs all three on the same countries and compares the labelings.** Where methods with different assumptions agree, the group is probably there. Where only one finds it, suspect the assumption
- **Other names you will meet:** hierarchical clustering (a tree of merges, no k until you cut it), spectral clustering (k-means on a graph), HDBSCAN (DBSCAN across densities), self-organizing maps (a grid of k-means centers, popular in oceanography)

Before you believe a clustering

Scaling decided the answer before the algorithm ran
Standardize, take logs where ranges are huge, scale a map's months together. Say what you did.

The number of groups is yours, defend it
Use the elbow, the silhouette, or BIC to rule out bad values. Then choose for a reason, and show the neighbors.

Check the seed
n_init above 1 for k-means and mixtures. If the labels change with random_state, you have not found anything.

Samples next to each other in time are not independent
Days in a regime, hours in a heat wave. Clusters of autocorrelated samples look sharper than they are.

Check it against something the algorithm never saw
The map, the calendar, the known physics, a different method. Agreement is the only evidence you get.

A cluster is a description, not a discovery
k-means cuts a continuum into pieces wherever you ask it to. The boundaries are where it cut, not where nature did.

- **For the assignment:** write one sentence for each box. What you scaled, why that k, whether the map changed with the seed, and what the zones correspond to on the Köppen map

Extra

Two reference slides. The math behind the three methods, and the scikit-learn calls in one place. Not covered in class.

The math behind the three methods

k-means minimizes the inertia, the within-cluster sum of squares

$$J = \sum_{i=1}^{n} \left\| x_i - \mu_{c(i)} \right\|^2$$

c(i) is the cluster of sample i and μ its center. Each step of the algorithm lowers J or leaves it alone: assigning to the nearest center cannot raise it, and the mean is the point that minimizes the squared distances. So it converges, to a local minimum.

A Gaussian mixture is a weighted sum of bell curves

$$p(x) = \sum_{k=1}^{K} \pi_k \, \mathrm{N}(x \mid \mu_k, \Sigma_k)$$

π_k are the shares and sum to 1. μ_k is a center and Σ_k a covariance matrix, which gives the ellipse its size and tilt. score_samples returns log p(x). The likelihood of the data is the product of p(x) over the samples.

BIC charges log n per parameter

$$\mathrm{BIC} = m \log n \; - \; 2 \log \hat{L}$$

m parameters, n samples, L̂ the maximized likelihood. Lower is better. AIC replaces log n with 2.

Expectation maximization, the E step: responsibilities

$$\gamma_{ik} = \frac{\pi_k \, \mathrm{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j \; \pi_j \, \mathrm{N}\left(x_i \mid \mu_j, \Sigma_j\right)}$$

γ_ik is the probability that sample i came from component k, given the current components. predict_proba returns it.

The M step: refit each component with the responsibilities as weights

$$N_k = \sum_i \gamma_{ik} \,, \qquad \pi_k = \frac{N_k}{n}, \qquad \mu_k = \frac{1}{N_k} \sum_i \gamma_{ik} \, x_i$$

$$\Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^{\mathrm{T}}$$

Alternate E and M until the log likelihood stops rising. Each round raises it or leaves it alone, so it converges to a local maximum. With γ_ik replaced by 0 or 1 for the nearest center, this is k-means.

Silhouette of one sample

$$s = \frac{b - a}{\max(a, b)}$$

a is the mean distance to the sample's own cluster, b the mean distance to the nearest other cluster.

Clustering in scikit-learn, in one place

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.cluster import KMeans, DBSCAN, MiniBatchKMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score, adjusted_rand_score

X = StandardScaler().fit_transform(feat)      # or MinMaxScaler((-1, 1))

km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
km.labels_                  # cluster of each training sample
km.cluster_centers_         # k by features, in scaled units
km.inertia_                 # the quantity it minimized
km.predict(X_new)           # nearest center for new samples
scaler.inverse_transform(km.cluster_centers_)   # read them

silhouette_score(X, km.labels_)                 # -1 to 1
adjusted_rand_score(labels_a, labels_b)         # agreement of two
labelings

db = DBSCAN(eps=0.6, min_samples=6).fit(X)
db.labels_                  # -1 is noise
db.core_sample_indices_     # which samples are cores
# no predict: fit KNeighborsClassifier on the cores if you need one

gm = GaussianMixture(n_components=4, n_init=5, random_state=0).fit(X)
gm.predict(X); gm.predict_proba(X)     # hard and soft labels
gm.means_, gm.covariances_, gm.weights_
gm.score_samples(X)         # log density, low = anomaly
gm.bic(X); gm.aic(X)        # lower is better
```

- **Scale first, always.** Every method here is a distance method
- **n_init:** 10 is the k-means default. For GaussianMixture the default is 1, so set it yourself
- **random_state:** set it, then change it once to see whether the answer moves
- **DBSCAN on a map:** metric="haversine" with latitude and longitude in radians, eps in radians
- **covariance_type:** "full" is the default and the most flexible. "diag" is cheaper and good enough when features are many. "tied" shares one shape across components, "spherical" is k-means with soft labels
- **A field becomes a table:** mask the NaNs, index with the mask, transpose to samples by features. Keep the masked latitude and longitude to plot the result
- **Discrete colors for labels:** ListedColormap, or cmap="tab10" with vmin and vmax set to the label range